

MKF-ADS: Multi-Knowledge Fusion Based Anomaly Detection System in Vehicular Control Area Networks

Pengzhou Cheng¹, Shouxuan Liu, Zongru Wu, Lixing Tan², and Gongshen Liu¹

Abstract—As the standard communication protocol for vehicular networks, the Controller Area Network (CAN) plays a crucial role in coordinating interactions among Electronic Control Units (ECUs). However, intruders can launch malicious attacks on the CAN due to the lack of security mechanisms. Intrusion Detection Systems (IDSs), as a defense component, are deployed to monitor the CAN and detect suspicious behavior resulting from an attack. Existing IDSs have high false alarms in complex attacks due to limited feature modeling capabilities. In this paper, we propose a multi-knowledge fusion-based anomaly detection system, called MKF-ADS. Specifically, the method designs a prediction-based framework that integrates a spatial-temporal correlation module with an attention mechanism (STcAM) and a patch-based sparse transformer (PatchST) module. The fine-pruned STcAM applies one-dimensional convolution (Conv1D) to extract spatial features and subsequently employs the Bidirectional Long Short Term Memory (Bi-LSTM) to capture temporal features, with an attention mechanism that focuses on important time steps. Meanwhile, the PatchST captures contextual features from independent univariate time series. Finally, the proposed model uses STcAM as a student model to learn intrinsic knowledge and cross-imitate PatchST's competencies through knowledge distillation. Extensive evaluations demonstrate that MKF-ADS is lightweight and effectively performs anomaly prediction and detection across six attack scenarios. Compared to the baseline, the proposed method is more robust, achieving a high F1-score while maintaining controllable false alarms.

Index Terms—Anomaly detection system, control area network (CAN), knowledge distillation, spatial-temporal contextual features.

I. INTRODUCTION

CONTROL Area Network (CAN) has become the standard communication protocol for vehicular networks, enabling seamless interaction among Electronic Control Units

(ECUs) [1]. The emergence of Intelligent Connected Vehicles (ICVs) has drawn increased attention to the CAN. To support interconnectivity, CAN has to open additional interfaces to enable intelligent services, such as Autonomous Vehicles (AVs), vehicular network communication, and cloud computing [2], [3], [4]. However, these intelligent services introduce severe cyberattacks to In-Vehicle Networks (IVNs) [5]. Unfortunately, the CAN bus was not designed with security mechanisms in mind, such as authentication and encryption [6]. When communicating via the bus or through external devices like Bluetooth or hardware ports [7], [8], the bus or ECUs can be hijacked, potentially leading to severe consequences for drivers and pedestrians [9], [10], [11]. Thus, securing vehicular networks has become a critical area of research [12].

Although encryption and authentication are commonly used to defend against cyberattacks [13], studies have shown that these methods demand substantial memory and computational resources [14], [15], [16] and account for backward compatibility [17]. In contrast, Intrusion Detection Systems (IDSs) offer an efficient mechanism for detecting vehicular cyberattacks and are categorized into two types, signature-based and anomaly-based [18], [19]. Signature-based IDS rely on supervised learning to detect known attacks from predefined databases. However, this method requires a substantial amount of labeled data, which is difficult to obtain for real-world CAN attacks. In contrast, anomaly-based IDSs (ADSs) are better suited to the open and complex vehicular network environment, as they can detect unknown malicious activities [20].

Recent research on ADSs has focused on modeling the correlations inherent in normal CAN messages to detect deviations indicative of malicious activity. For instance, Song et al. [1] proposed an anomaly detection method that identifies attack messages by analyzing noisy pseudo-normal data. However, it requires labeled abnormal datasets during training. In contrast, Talyor et al. [21] and Qin et al. [22] applied long and short-term memory (LSTM) models to learn the temporal correlations of CAN transition bits to detect deviations between abnormal and normal messages. However, they model all bits (typically 64 bits) without considering their actual meaning and show poor predictive performance. Subsequently, Convolutional Neural Networks (CNNs) and Attention Mechanisms (AM) are introduced to capture spatial-temporal features [23], [24]. However, they struggle to capture long-term historical dependencies

Received 24 September 2024; accepted 20 April 2025. Date of publication 8 May 2025; date of current version 19 September 2025. This work was supported in part by the Joint Funds of the National Natural Science Foundation of China under Grant U21B2020 and in part by the National Natural Science Foundation of China under Grant 62406188. The review of this article was coordinated by Prof. Abbas Jamalipour. (Corresponding author: Gongshen Liu.)

Pengzhou Cheng, Zongru Wu, and Gongshen Liu are with the Department of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: pengzhouchengai@gmail.com; wuzongru@sjtu.edu.cn; lgshen@sjtu.edu.cn).

Shouxuan Liu is with the College of Engineering, University of Michigan, Ann Arbor, MI 48105 USA (e-mail: maksiml@umich.edu).

Lixing Tan is with the College of Information Engineering, Taizhou University, Taizhou 225300, China (e-mail: tanlixing@nuaa.edu.cn).

Digital Object Identifier 10.1109/TVT.2025.3564575

in time-series data. Thus, Alkhatib et al. [25] developed the CAN-BERT model to capture the contextual features of CAN ID sequences. Similarly, Nam et al. [26] regarded the CAN ID sequence as a sentence composed of multiple words and applied the Generative Pretrained Transformer (GPT) model to learn the normal pattern of each sequence. However, transformer-based models are unsuitable for IVNs with limited resources due to their large number of parameters.

To address the above challenges, we propose a **Multi-Knowledge Fusion based Anomaly Detection System**, called MKF-ADS. This model is lightweight and follows the multivariate time series prediction paradigm. First, we apply the “READ” algorithm [27] to extract meaningful signal boundaries and continuous physical signals from CAN messages. These signals are then fed into a multi-knowledge feature extract module, which incorporates spatial-temporal correlations and contextual information to perform signal prediction. Anomaly detection is conducted by evaluating the deviation between actual and predicted signals. To accommodate the resource constraints of IVNs, the fusion mechanism employs a top-down guidance and cross-imitation approach based on knowledge distillation. Extensive experiments show that the proposed model is effective and robust against six attack scenarios. Compared to the baseline in this field, our method achieves a competitive F1-score. The main contributions of this paper are as follows.

- 1) We propose MKF-ADS, a knowledge fusion-based anomaly prediction framework. The framework integrates the STcAM module to model spatial-temporal correlations and the PatchST module to capture long-term contextual dependencies.
- 2) We design a lightweight STcAM module, where the Conv1D extracts spatial knowledge, BiLSTM captures temporal knowledge, and the soft attention mechanism focuses on important time steps. Additionally, PatchST is introduced to separate multivariate time series and capture contextual knowledge across all time steps.
- 3) We construct a multi-knowledge fusion framework, with STcAM as the student model and PatchST as the teacher model. STcAM not only learns inherent knowledge but also cross-mimics valuable knowledge from PatchST, helping to resolve conflicts between different knowledge sources and enhancing training stability.
- 4) We evaluate the prediction and detection performance of MKF-ADS in six intrusion scenarios and verify its generalization on various CAN IDs, multi-time steps, and real-world attacks from ROAD tasks. Also, we assess the model’s feasibility in terms of complexity and overall efficiency when deployed in vehicular networks.

The rest of this paper is organized as follows. Section II reviews and discusses the related works. Section III introduces preparatory knowledge on IVNs, attack models, and design intuition. Section IV describes the MKF-ADS framework. Section V presents the experimental results and discusses the performance and efficiency advantages. Finally, Section VI concludes the work.

II. RELATED WORK

This section provides an in-depth overview of previous works on vehicular network security, broadly categorized into authentication-based, cryptography-based, and IDS-based.

A. Security Models for Vehicular Network

Authentication-based and cryptography-based methods have been extensively studied by several researchers [44]. Dariz et al. [45] proposed a method that combines encryption and authentication for vehicular networks, while Farag et al. [46] used a dynamic symmetric key to encrypt the payload without modifying the CAN ID. Although these methods offer some level of security for IVNs, they are prone to response delays or increased bus loads [47]. Alladi et al. [48] developed a lightweight authentication and attestation scheme to secure vehicles in transit. Similarly, Cui et al. [49] introduced a lightweight encryption and authentication scheme for the CAN bus based on message authentication codes and the Grain stream cipher. However, these methods are either computationally demanding or require protocol modification, limiting their ability to provide reliable and efficient defense.

IDSs have been widely studied in vehicular networks [50]. Early IDSs detect malicious messages using predefined knowledge bases [28], [29], [30], [31], yet they exhibit high false alarms and fail to detect unknown attacks effectively. Although statistics-based IDSs are efficient and lightweight, they may overlook low-volume and aperiodic attacks. Machine learning-based methods have also been extensively researched [32], [33]. However, these approaches face a bottleneck in detection performance. With the growing density of vehicular data and enhanced computing resources, deep learning-based strategies have emerged. Song et al. [34] proposed a deep CNN-based method for intrusion detection, eliminating the need for hand-crafted feature extraction. Xun et al. [35] and Luo et al. [36] introduced CNN and LSTM-based models to further improve feature extraction. Cheng et al. [37] incorporated attention mechanisms to reduce false alarms, while Javed et al. [38] replaced LSTM with GRU to enhance inference speed. Although these methods improve detection capabilities, they rely heavily on large amounts of labeled data for supervised learning, making them impractical against attack variants.

In contrast, ADSs can detect unknown attacks [12]. Ashraf et al. [9] utilized an auto-encoder (AE) to reconstruct input messages using spatial-temporal features. Agrawal et al. [2] proposed a method that reconstructs sequence directly using spatial-temporal features and detects anomalies based on threshold deviations. Jeong et al. [39] introduced an explainable intrusion detection based on a self-supervised AE paradigm. Tariq et al. [40] proposed CANTransfer, a Convolutional LSTM-based transfer learning model. Zhang et al. [41] introduced graph CNN-based ADS to detect falsification attacks. Given that CAN messages always follow fixed patterns, ADS also adopts a prediction-based paradigm to detect cyberattacks. Taylor et al. [21] and Qin et al. [22] developed LSTM-based models to predict the next package, with error measured using the log loss function on the predicted output. Zhang et al. [42] combined CNN with a Gated

TABLE I
COMPARISON THE PROPOSED MKF-ADS TO EXISTING SCHEMES

Methods	[29], [30] [31], [32]	[33], [34]	[35], [36], [37] [38], [39]	[2], [40], [41] [9], [42]	[25], [22], [23] [43], [44]	[26], [27]	MKF-ADS (Ours)
Categories	SL	ML	DL-based supervised-learning	DL-based anomaly classification	DL-based anomaly prediction	Transformer-based DL	DL-based anomaly prediction
Technology	Entropy, frequency and optimal	HMM, SVM, and XGBoost	CNN, LSTM, GRU, and AM	CNN, LSTM, and One-class	CNN, LSTM, and AM	BERT and GPT	Multi-knowledge fused
Attack threats ¹	Low	Low	Medium	Medium	High	Medium	High
Knowledge modeling	✗	✗	✓	✓	✓	✗	✓
Computation complexity ²	Low	Low	High	Medium	Medium	High	Low
Collection delay ³	Low	Low	High	High	Low	High	Low
Detection Capability (1-5) ⁴	★	★★	★★★	★★★★	★★★★	★★★★★	★★★★★

¹ Low: Synthetic attacks. Medium: Real-world attacks. High: Complex real-world attacks;

² Low: Lightweight and designed for fast training and inference. Medium: Suitable for IVNs. High: Evaluated only on PCs;

³ Low: Preprocessing is simple. High: Preprocessing relies on complex algorithms;

⁴ ★: The method demonstrates poor detection performance when evaluated on synthetic attacks. ★★: The method encounters performance bottlenecks. ★★★: It effectively detects known attacks but exhibits weak performance on unknown attacks. ★★★★: The method excels in contextual analysis and surpasses others in anomaly detection. ★★★★★: It shows effective and robust detection performance, particularly in complex attack scenarios.

Recurrent Unit (GRU) to model spatial-temporal features. Building on this, Mansouria et al. [43] introduced Gaussian Naive Bayes (GNB) to improve detection performance. Sun et al. [24] incorporated an attention mechanism (AM) to enhance adaptive learning across time steps. Interestingly, Alkhatib et al. [25] and Nam et al. [26] suggested applying state-language models like BERT and GPT to ADS. Although these models incorporate rich contextual features, they are not suitable for IVNs due to their resource-intensive demands.

B. Comparison In-Depth for IVN-IDS

Based on modeling strategies and objectives, these works are categorized into Statistical Learning (SL), Machine Learning (ML), and Deep Learning (DL). Table I presents the main characteristics and comparisons between MKF-ADS and existing works. Intuitively, early research based on SL and ML shares common characteristics, including fixed feature extraction patterns [28], [30], [51] and worrisome performance [32]. In contrast, most studies are DL-based, which pursue different objectives, such as using supervised learning to identify known attacks and anomaly-based techniques to classify or predict unknown attacks. We find that supervised DL is significantly enhanced by rich feature extraction. For example, CNNs extract spatial features [34], LSTM captures temporal features [36], and attention mechanism models valuable features [37]. However, these models have notable drawbacks, such as higher computation complexity and a limited detection range. In anomaly classification, reconstruction modeling calculates the deviation score of normal messages [2], [9], but this technique suffers from significant data collection delays. Also, the one-class classifier, a minimal adaption, often sacrifices its ability to detect known attacks [40], [41].

In contrast, we tend to regard anomaly detection in IVN as a time-series prediction problem due to its lower collection delay and ability to resist a wider range of attacks. Traditional LSTM-based methods initially prove these advantages [21], [22]. Later, valuable feature extraction was recognized as equally important, but it introduced similar complexity as the previous branch due to the additional encoding components for spatial-temporal

modeling [24], [42]. From a knowledge modeling perspective, SL and ML models rely on simple, shallow detection criteria. Existing DL-based studies show that spatial-temporal knowledge is effective for IDS. Moreover, the contextual knowledge extracted by transformer-based models has proven to offer comparable, if not superior, performance to spatial-temporal ones [25], [26]. However, it is impractical in IVNs due to the exponential growth in computational complexity. Importantly, most research to date has been primarily confined to simulation testing on workstations [34], [51], with only a few works [7], [30], [37] venturing into vehicle-level testing or real-time analysis.

Given the complementarity of spatial-temporal and contextual knowledge, the multi-knowledge fused is a practical approach. Therefore, the proposed MKF-ADS introduces a cross-knowledge distillation framework, where contextual knowledge serves as the teacher and spatial-temporal knowledge as the student. This structure allows the model to capture subtle deviations by enriching knowledge, thereby identifying potential threats. To ensure real-time detection, we apply fine-pruning when modeling spatial-temporal knowledge, with reduced complexity guided by contextual knowledge.

III. PRELIMINARIES

In this section, we provide a brief overview of vehicular communication. Next, we analyze in-vehicle vulnerability and present various attack scenarios. We then discuss the design intuition based on observation from the collected CAN logs.

A. Control Area Network Bus

The CAN bus is the de facto communication standard for vehicular networks, organizing ECUs into distinct domains such as safety, powertrain, and comfort. Each domain handles specific tasks, including automatic crash notification, vehicle diagnostics, and infotainment services [52]. These ECUs are interconnected through a central gateway and transmit messages at fixed intervals on the CAN bus. Fig. 1 illustrates the standard structure of the CAN data frame. The arbitration mechanism—using 11 bits for CAN 2.0 A and 29 bits for CAN 2.0 B—determines

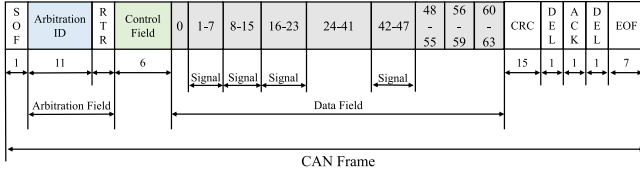


Fig. 1. The standard structure of CAN frame. signals associated with each ID are processed using the “READ” method. For instance, we extracted four effective physical signals on the ‘0x260’ ID, including $1^{th} - 7^{th}$, $8^{th} - 15^{th}$, $16^{th} - 23^{th}$, and $42^{th} - 47^{th}$.

the priority of message transmission. The control and data fields represent transmission length and signal values, respectively.

To support intelligent services, additional external interfaces that communicate via WIFI, Bluetooth, and 5G are opened up, which increases the potential security risks. This study focuses on the arbitration and data fields of CAN frames to construct ADS, as they are frequently targeted in cyberattacks.

B. Attack Model

In this study, we assume the attacker can remotely compromise one or more ECUs through a wireless interface (e.g., a telemetries port [37]) or physically (e.g., via OBD-II [53]). Thus, each ECU may serve as an entry point for the attacker to access the CAN bus and launch various attacks. Drawing from car-hacking [1], ROAD [17], and the attacks described in work [23], we construct the following attack scenarios:

- **Denial of Service (DoS) Attack:** This attack aims to interfere with the intended function of the CAN bus. The attacker usually sends a flood of higher-priority, redundant messages to exhaust system resources and paralyze the functions.
- **Fuzzy Attack:** This attack injects randomly generated CAN IDs and data frames into compromised ECUs. The randomly generated CAN IDs range from 0x000 to 0x7ff, including those originally extracted from the vehicle.
- **Suspension Attack:** This attack aims to prevent compromised ECUs from transmitting CAN messages.
- **Replay Attack:** This attack hijacks a valid message at a specific time and transmits it later. The attacker usually exploits this to gain illegitimate authority.
- **Spoofing Attack:** This attack first sniffs valid messages from a specific ECU and then impersonates the compromised ECU by mimicking its message transmission frequencies with forged signals.
- **Masquerade Attack:** This attack is more insidious as the legitimate messages will be deleted before injecting a spoofing message [54].

These attack mechanisms align with those described in work [24], which categorizes them as either message injection or falsification. First, message injection attacks (e.g., DoS, Suspension, and Replay) are usually related to message frequency or the statistical sequence of CAN ID, so the proposed IDS should be sensitive to temporal knowledge. Second, message falsification attacks (e.g., Fuzzy, Spoof, and Masquerade) are generally related to message content, so the proposed IDS

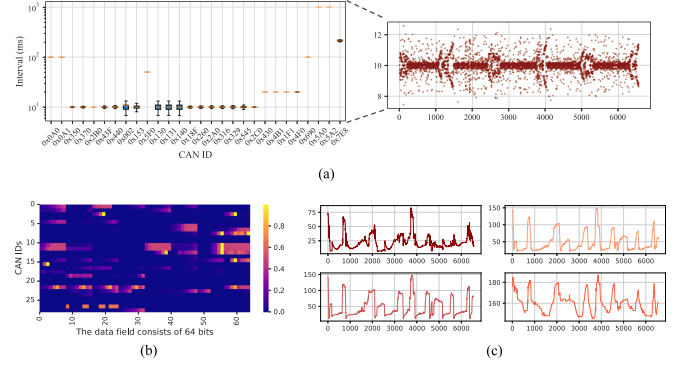


Fig. 2. Illustration of the design intuition for knowledge modeling, where (a) shows the frequency of CAN IDs with an example; (b) depicts the bit flip rate for each CAN ID; and (c) visualizes valid signals based on CAN sequences.

should effectively analyze spatial knowledge. Moreover, context knowledge, such as semantic features, is more robust and can enhance knowledge modeling. Thus, MKF-ADS caters to these requirements, improving the effectiveness and efficiency of detecting malicious messages.

C. Design Intuition for Knowledge Fusion

Knowledge fusion-based modeling is crucial for improving ADS performance. Previous studies have highlighted the importance of temporal [21], [22] and spatial features [34], [35], [36] as well as spatial-temporal features [37], [43]. In addition, contextual knowledge exhibits a positive contribution to ADS [25]. To highlight the contribution of knowledge fused contribution, we conduct statistical analysis. In Fig. 2(a), CAN sequences transmit messages according to predefined frequencies (e.g., CAN ID=‘260’ has a fixed frequency of 10 ms), which requires our ADS to capture temporal features over both long-time and short-distance. Empirically, feature extraction is generally handled by recurrent neural networks (RNNs) due to their temporal capture capabilities. Besides, Fig. 2(b) shows dramatic bit flip changes in localized regions. Thus, the proposed ADS should integrate CNNs to capture significant spatial features across all extracted signals. Also, transformer-based components are highly effective at extracting semantic features, making them one of the most powerful tools for ADS. Fig. 2(b) highlights the vertical contextual relationships between signals, while the trends in Fig. 2(c) illustrate horizontal interactions between valid signals. Although this approach significantly improves performance, it also introduces higher complexity [25], [26].

To this end, we propose MKF-ADS, which also utilizes a CNN-LSTM paradigm to model spatial-temporal knowledge. In contrast, the proposed model’s parameters are significantly reduced through fine-pruning. Meanwhile, the feature dimensional is reduced by the “READ” algorithm. For example, CAN ID=‘0x260’ has four valid signals, as shown in Fig. 1. Importantly, we consider semantics as high-level knowledge capable of enhancing spatial-temporal knowledge through knowledge distillation. In short, MKF-ADS ensures both the richness of knowledge and low complexity.

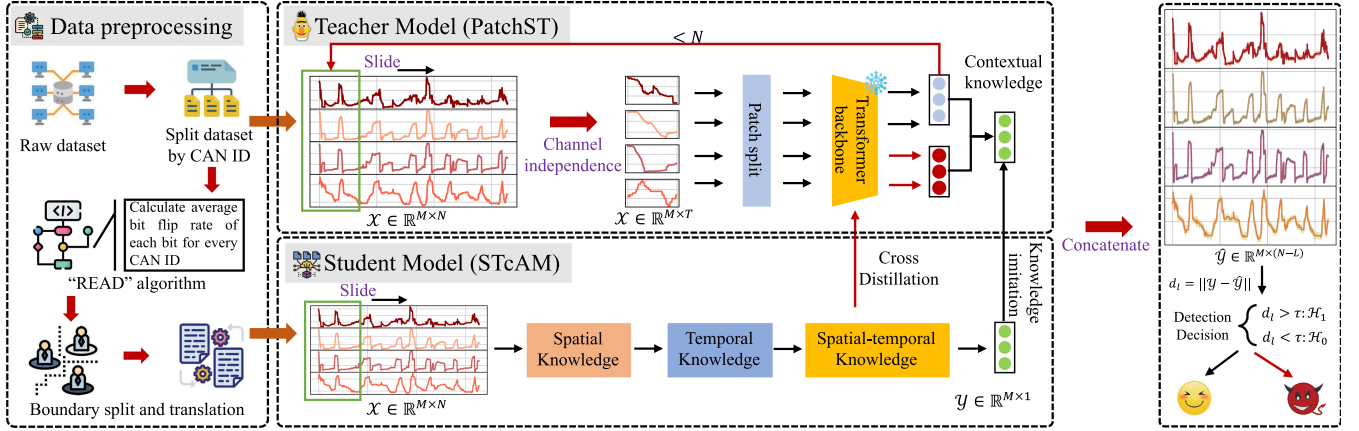


Fig. 3. Overview of the proposed MKF-IDS.

IV. METHODOLOGY

In this section, we present MKF-ADS, consisting of the spatial-temporal knowledge module (STcAM), contextual knowledge module (PatchST), a multi-knowledge fused module, and an anomaly detection module, as shown in Fig. 3.

A. Overview

MKF-ADS aims to enhance the overall performance of in-vehicle ADS in predictive paradigms. The improvements span the entire pipeline and are described as follows.

- *Data preprocessing*: The physical payload is analyzed using bit flip rate and boundary decision algorithms to extract signals and improve training efficiency.
- *STcAM*: The preprocessed data are fed into the STcAM to extract spatial and temporal knowledge, while the soft attention mechanism highlights significant timestamps.
- *PatchST*: The module extracts fine-grained contextual knowledge from independent signals using a patch-based transformer model.
- *Cross Knowledge Distillation*: This module allows STcAM to learn internal knowledge maximally, and continuously mimic contextual knowledge from PatchST.
- *Anomaly Detection*: This module uses the deviation between the predicted and actual signals to determine the optimal threshold for detecting anomalies.

In the online detection phase, the CAN bus is flagged as compromised if the deviation between predicted and received values exceeds the threshold.

B. Data Preprocessing

First, we evaluate the proposed model on the car hacking for intrusion detection dataset,¹ provided by the hacking and countermeasure research lab (HCRL). The dataset was collected via the OBD-II port of real-world vehicles. It contains crucial information for CAN messages, including the Timestamp, CAN ID, DLC, and DATA[0]-DATA[7]. To accommodate DL-based

model input, we apply a prediction-based paradigm for data preprocessing, as shown in Fig. 3. Specifically, we split the payload of CAN messages with the same ID from the raw dataset. Then, we extract significant physical signals using the “READ” and boundary split algorithms [27], without relying on the CAN communication protocol. For instance, the valid signal for CAN ID=‘260’ is reduced from the original 64 bits to 41 bits. Following this, we introduce a feature generator that processes continuous raw CAN messages as multivariate time series and generates discrete sub-sequences. The feature generator performs two key functions: i) it uses a sliding window T ($T \in \mathbb{N} | T > 10$) to aggregate recent messages and produce a two-dimensional (2-D) vector; ii) it translates the 2-D vector into decimal format. To increase the numerical stability and accelerate model convergence, the min-max normalization function is used to normalize the dataset features, calculated as follows:

$$X_{\text{normalized}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}, \quad (1)$$

where X is raw data, $X_{\min}$ is the minimum value within the dataset for a specific feature, and $X_{\max}$ is the maximum value. After normalization, all features are scaled to similar value ranges between zero and one. Notably, we only considered the CAN ID from crucial functional areas, as they have fixed communication frequency (e.g., 10 ms). Next, we parameterize MKF-ADS as $\mathcal{M}_\theta = \mathcal{M}_\theta(X, Y, ID)$, where $X(f_1, \dots, f_M) = \{X(f_1), X(f_2), \dots, X(f_M)\} \in \mathbb{R}^{M \times T}$ donates a training instance. Here, $X(f_i)$ donates the i -th physical signal, which contains the t -th observation as determined by the sliding window. $Y = \{y_1, y_2, \dots, y_s\} \in \mathbb{R}^{M \times 1}$ is the corresponding label, and $\mathcal{M}$ is a nonlinear mapping function.

To evaluate robustness, we also evaluate MKF-ADS on the Real ORNL Automotive Dynamometer (ROAD) CAN intrusion datasets.² Compared to the HCRL dataset, this dataset offers a more comprehensive set of attack scenarios, including fuzzing, spoofing, and masquerade attacks. It is noted that its data preprocessing is performed using a feature generator following the same criteria.

¹<https://ocslab.hksecurity.net/Datasets/CAN-intrusion-dataset>

²<https://0xsam.com/road/>

C. Proposed MKF-ADS

Once the data meets the input requirements of the proposed model, it is fed into the model to generate prediction signals. The detailed design of MKF-ADS is as follows:

1) *STcAM Module*: The module is designed to be lightweight compared to previous works while maintaining detection performance. Given the multivariate time series for the i -th time window, we first adopt a one-dimensional CNN layer to extract spatial knowledge, as calculated below:

$$s_t^j = h_t^j(f) * x_t^j(f) = \sum_{m=0}^N h_t^j(f-m) x_t^j(m), \quad (2)$$

where s_t^j represents the t -th spatial feature in the j -th channel for convolution, $h_t^j(f)$ donates the convolution kernel, $x_t^j(f)$ refer to the feature map, and f is the kernel size (3×1). To nonlinearize spatial features, the rectified linear unit (ReLU) function is used to retain positive features and discard negative ones, as calculated by: $z(s) = s^+ = \max(0, s)$. Next, we utilize a bidirectional LSTM (BiLSTM) to capture temporal knowledge within the window size and reduce timing corruption from window sliding. The key function of the LSTM in each direction is to preserve information through the cell state. Thus, the latent representation h , which captures the spatial-temporal relationship, is calculated as follows:

$$h_t = o_t * \sigma_1(c_t), \quad (3)$$

where o_t and c_t are the output gate and cell state activation values, respectively, and σ_1 is the Tanh activation function. The cell state is influenced by the forget gate f , input gate i , and output gate o on current moment t , calculated as follows:

$$\begin{aligned} f_t &= \sigma_2(W_f[h_{t-1}, x_t], b_f), \\ i_t &= \sigma_2(W_i[h_{t-1}, x_t], b_i), \\ o_t &= \sigma_2(W_o[h_{t-1}, x_t], b_o), \end{aligned} \quad (4)$$

where W and b are weights and biases learned during model training, and σ_2 is the Sigmoid activation function. Hence, the cell state c_t is generated by retaining information from the previous moment c_{t-1} , via the forget gate f_t and updating the current state $\tilde{c}_t$, via the input gate i_t , calculated as follows:

$$\begin{aligned} \tilde{c}_t &= \sigma_1(W_c[h_{t-1}, x_t], b_c), \\ c_t &= (f_t * c_{t-1}) + (i_t * \tilde{c}_t). \end{aligned} \quad (5)$$

After obtaining the forward spatial-temporal knowledge $(\vec{h}_1, \dots, \vec{h}_t)$ and backward spatial-temporal knowledge $(\overleftarrow{h}_1, \dots, \overleftarrow{h}_t)$, we concatenate them to form the final output: $h_i = [\vec{h}_i; \overleftarrow{h}_i]$. Note that all spatial-temporal components share parameters. Thereafter, we introduce the soft attention mechanism to assign adaptive weights to the spatial-temporal representation, allowing the model to focus on significant time steps, calculated as follows:

$$\alpha_i = \frac{\exp(h_i)}{\sum_{i=1}^{t-1} \exp(h_i)}, \quad (6)$$

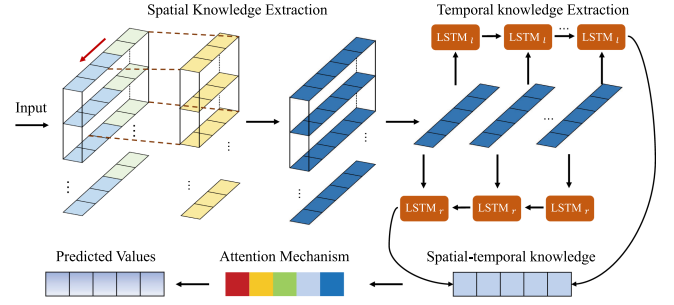


Fig. 4. Structure of STcAM component.

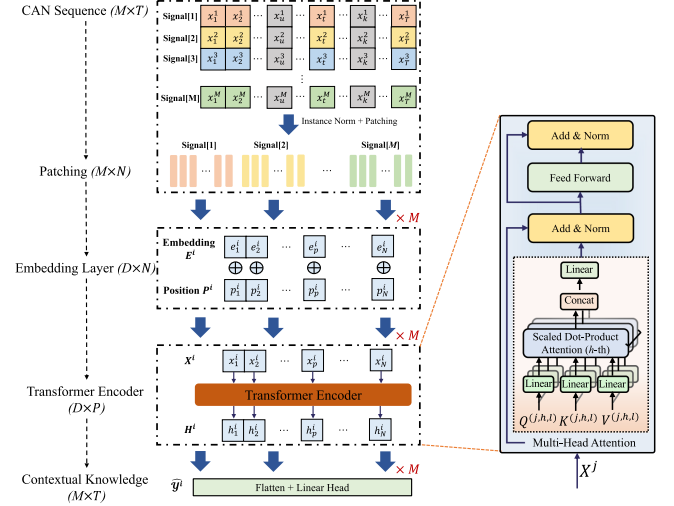


Fig. 5. Structure of PatchST component.

where α_i is the conditional probability distribution, computed using the softmax function. The attention vector of spatial-temporal knowledge is presented as follows:

$$h^* = \sum_{i=1}^{t-1} \alpha_i h_i. \quad (7)$$

To avoid overfitting, the dropout layer is introduced. Finally, the output vector is fed into a dense layer to predict the future time step. The STcAM component is shown in Fig. 4.

2) *PatchST Module*: Transform-based contextual knowledge is one of the state-of-the-art approaches for anomaly detection in multivariate time series. Inspired by [55], we introduce PatchST as a complement to spatial-temporal knowledge, alleviating the challenges of using complex models in resource-constrained IVNs [25], [56].

Given a multivariate time series with lookback window T : $(x_1, \dots, x_T)$, where each x_t at time step t is a vector of dimension M , we extract the context knowledge to predict future physical signals. PatchST is illustrated in Fig. 5. First, we split the CAN sequence $X \in \mathbb{R}^{M \times T}$ into M univariate series $x^{(i)} \in \mathbb{R}^{1 \times T}$, where each series of length T starts at time index 1 and slides across the window size T . Each univariate series is independently fed into the transformer backbone according to the channel-independence setting. The transformer-based model then generates the prediction $\hat{y}^{(i)}$.

Algorithm 1: Optimized MKF-ADS Algorithm.

Input: Dataset X_D, X_{test} ; STcAM, PatchST
Output: $res = []$, MKF-ADS;

```

1  ## Offline Training Phase:
2  while PatchST not converged do
3      for  $X \in X_D$  do
4           $X_p \leftarrow \text{Patching}(X)$ 
5           $X_d \leftarrow \text{Embedding}(X_p)$ 
6           $\hat{y} \leftarrow \text{PatchST}(X_d)$ 
7           $\mathcal{L} \leftarrow \text{MAE}(y, \hat{y})$ 
8           $\mathcal{L}.\text{backward}()$ 
9      end
10 end
11 while MKF-ADS not converged do
12     Initialization: PatchST.eval()
13     for  $X \in X_D$  do
14          $\hat{y}_t \leftarrow \text{PatchST}(X)$ 
15          $\hat{y}_s, X_a \leftarrow \text{STcAM}(X)$ 
16          $\hat{y}_{\text{cross}} \leftarrow \text{PatchST}(X_a)$ 
17          $\mathcal{L} \leftarrow \alpha \mathcal{L}_{\text{pred}}(y_s, \hat{y}_s) + \beta \mathcal{L}_{\text{CKD}}(\hat{y}_t, \hat{y}_{\text{cross}})$ 
18          $\mathcal{L}.\text{backward}()$ 
19     end
20 end
21  $\tau \leftarrow \text{Calculate Threshold using Eqn. 16}$ 
22 ## Online Detection Phase:
23 foreach  $X \in X_{test}$  do
24      $\hat{y} \leftarrow \text{MKF-ADS}(X)$ 
25      $\mathcal{L} \leftarrow \text{MAE}(y, \hat{y})$ 
26      $res.append(\text{metrics}(\mathcal{L}, \tau))$ 
27 end

```

Patching: Given a univariate time series $x^{(i)}$, we first divide it into patches $x_p^{(i)} \in \mathbb{R}^{P \times N}$, where P is the patch length, N is the number of patches, $N = \lfloor \frac{(L-P)}{S} \rfloor + 2$, and S is the stride between two consecutive patches. This reduces the number of input tokens from T to approximately T/S , which decreases the attention map size quadratically by a factor of S . In other words, PatchST is sparse, focusing on both local and longer historical sequences simultaneously, significantly improving forecasting performance.

Embedding: These patches are projected into the Transformer latent space of dimension D using a trainable linear projection $W_p \in \mathbb{R}^{D \times P}$, and a learnable additive position encoding $W_{pos} \in \mathbb{R}^{D \times N}$, calculated as follows:

$$e_t^{(i)} = W_e x_p^{(i)} + b_e,$$

$$p_t^{(i)} = W_{pos} x_p^{(i)} + b_{pos}, \quad (8)$$

where $e_t^{(i)}$ and $p_t^{(i)}$ represent the embedding feature and positional embedding, respectively. Thus, the model input could present $x_d^{(i)} = e_t^{(i)} + p_t^{(i)}$, where $x_d^{(i)} \in \mathbb{R}^{D \times N}$ is fed into the Transformer encoder.

Transformer encoder: We use a vanilla Transformer encoder to extract contextual knowledge, which consists of two sub-layers: a multi-head attention mechanism and a position-wise

feed-forward network. Additionally, a residual connection is applied between the two sub-layers, followed by layer normalization. For the l -th transformer encoder, each multi-head attention function h transforms $x_d^{(i)}$ into query matrix $Q_h^{(i)} = (x_d^{(i)})^T W_h^Q$, key matrix $K_h^{(i)} = (x_d^{(i)})^T W_h^K$, and value matrix $V_h^{(i)} = (x_d^{(i)})^T W_h^V$, where $W_h^Q, W_h^K \in \mathbb{R}^{D \times d_k}$ and $W_h^V \in \mathbb{R}^{D \times D}$. The attention output $O_h^{(i)} \in \mathbb{R}^{D \times N}$ is then calculated as follows:

$$O_i^{(i)} = \sigma \left(\frac{Q_h^{(i)} K_h^{(i)T}}{\sqrt{d_k}} \right) V_h^{(i)}, \quad (9)$$

where σ is the softmax function. Then, all $O_h^{(i)}$ are concatenated column-wise and projected into the representation space via a linear layer. The final multi-head attention for the l -th transformer encoder is then calculated as:

$$X_A^{(i,l)} = W^A [O^{(i,l)_1}, \dots, O^{(i,l)_h}, \dots, O^{(i,l)_H}], \quad (10)$$

where W^A is the attention output weight, and H represents the total amount of the self-attention module.

Next, a residual connection and layer normalization are applied to each sub-layer of the transformer encoder. The residual connection enables multi-transformer layers to be stacked without degradation. Besides, layer normalization ensures that the input to each neuron has a consistent mean and variance, helping to accelerate the network convergence. The output representations are then calculated as follows:

$$X_N^{(i,l)} = L(X^{(i,l)} + X_A^{(i,l)}) = L(X^{(i,l)} + A(X^{(i,l)})), \quad (11)$$

where $A(\cdot)$ represents the output function of the multi-head attention module, and $L(\cdot)$ denotes the layer normalization function, calculated as follows:

$$L(X^{(i,l)}) = \text{ReLU} \left(\frac{g^m}{\sigma^m} (X^{(i,l)} - \mu^l) + b \right), \quad (12)$$

where g and b represent the gain and bias parameters, respectively. The mean and variance of the m -th layer are given by $\mu^m = \frac{1}{N} \sum_{i=0}^N X^{(i,l)}$ and $\sigma^m = \sqrt{\frac{1}{N} \sum_{i=0}^N (X^{(i,l)} - \mu^l)^2}$, respectively.

Although $X_A^{(i,l)}$ captures the contextual relationships of the messages at each patch, it performs only a matrix multiplication, i.e. a linear transformation. To satisfy non-linear, a position-wise feed-forward with two liner layers is applied to compute nonlinear attention features:

$$X_F^{(i,l)} = W_{f_2} \sigma(W_{f_1} X_N^{(i,l)} + b_{f_1}) + b_{f_2}, \quad (13)$$

where the W_{f_1}, W_{f_2} and b_{f_1}, b_{f_2} are the weight and bias of the module, respectively, σ refers to the ReLU activation function. Immediately, $X_F^{(i,l)}$ is fed into the second normalization layer, resulting in the final contextual features $H^{(i,l)} = \{h_1^{(i,l)}, \dots, h_p^{(i,l)}, \dots, h_N^{(i,l)}\}$ of the l -th encoder. After forwarding through all transformer layers, a flattening layer followed by a linear head is employed to produce the predicted physical signal $\hat{y}^{(i)}$.

3) *Cross Knowledge Distillation:* It is impractical to apply Transformer-based models directly to IVN-ADS. To address

this challenge, we introduce cross-knowledge distillation [57], enabling knowledge fusion from PatchST to STcAM, while avoiding knowledge conflict during training. Given the intermediate output of STcAM, denoted as X_A (e.g., the output of attention mechanism), the cross-head student prediction $\hat{y}_{\text{cross}}$ is generated by a share component of PatchST. The CKD is formulated as follows:

$$\mathcal{L}_{\text{CKD}} = \text{KL}(\hat{y}_t || \hat{y}_{\text{cross}}) = \frac{1}{N} \sum_{i=0}^N \hat{y}_{\text{cross}}^{(i)} \log \frac{\hat{y}_{\text{cross}}^{(i)}}{\hat{y}_t^{(i)}}, \quad (14)$$

where $\text{KL}(\cdot)$ represents the Kullback–Leibler (KL) divergence, and $\hat{y}_t$ represents the prediction of the teacher model. The final optimization objective for the offline training phase is computed as follows:

$$\begin{aligned} \mathcal{L} &= \alpha \mathcal{L}_{\text{pred}} + \beta \mathcal{L}_{\text{CKD}} \\ &= \alpha \frac{1}{N} \sum_{i=0}^N |y_s^{(i)} - \hat{y}_s^{(i)}| + \beta \frac{1}{N} \sum_{i=0}^N \hat{y}_{\text{cross}}^{(i)} \log \frac{\hat{y}_{\text{cross}}^{(i)}}{\hat{y}_t^{(i)}}, \end{aligned} \quad (15)$$

where $\mathcal{L}_{\text{pred}}$ is the Mean Average Error (MAE) function, α and β represent the weight of the two optimization tasks, respectively.

In the online detection phase, we use Root Mean Square Error (RMSE): $\sqrt{\frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2}$ and Mean Absolute Percentage Error (MAPE): $\frac{1}{N} \sum_{i=1}^N \frac{|y^{(i)} - \hat{y}^{(i)}|}{y^{(i)}}$ to evaluate the prediction performance.

4) *Anomaly Detection*: In this phase, we establish an anomaly detection threshold using a confidence interval-based approach. The threshold is the right-side boundaries of the K -sigma confidence interval derived from the prediction error distribution, calculated as follows:

$$\text{Threshold}(\tau) = \mu_{\text{normal_messages}} + \mathbf{K} * \sigma_{\text{normal_messages}}, \quad (16)$$

where $\mu_{\text{normal_messages}}$ is the mean of the prediction error distribution of the normal messages, and $\sigma_{\text{normal_messages}}$ is the standard deviation for the same distribution. The value of K and the threshold are determined by an in-depth analysis in Section V-B. The proposed model is described in Algorithm 1.

D. Runtime Complexity

In this subsection, we provide a theoretical evaluation of the temporal and spatial complexity of MKF-ADS.

1) *Temporal Complexity*: During offline training, the proposed model learns both spatial-temporal knowledge from STcAM and contextual knowledge from PatchST. The time complexity of the PatchST O_p is calculated as follows:

$$\begin{aligned} O_p + \partial O_p &= 2O_p = \underbrace{2O(N \cdot p \cdot e)}_{\text{Embedding Layer}} \\ &+ \underbrace{2O(l \cdot (N^2 \cdot e + N \cdot e^2))}_{\text{Transformer Layer}} + \underbrace{2O(e \cdot p \cdot d_u)}_{\text{Dense Layer}}, \end{aligned} \quad (17)$$

where e is the embedding size, p is the patch size, N is the number of patches, l is the transformer layers, and d_u is the dense units. Thus, the temporal complexity of the MKF-ADS O_m is composed of O_p in the test phase and the STcAM module

O_s , denoted as follows:

$$\begin{aligned} O_m &= O_s + \partial O_s + O_p = 2O_s + O_p = \underbrace{2O(c^2 \cdot f_s \cdot s_l)}_{\text{Conv1D}} \\ &+ \underbrace{2O(s_l \cdot h_s \cdot c)}_{\text{BiLSTM}} + \underbrace{2O(s_l \cdot 2h_s^2 + s_l)}_{\text{Attention Mechanism}} + \underbrace{2O(d_u)}_{\text{Dense Layer}} + O_p, \end{aligned} \quad (18)$$

where s_l is the sequence length, f_s and c are the filter size and the number of channels in the Conv1D, respectively, h_s is the hidden size in the LSTM, and d_u is dense units. Thus, in the training phase, the total temporal complexity of MKF-ADS is determined by N , e , c , and h_s . In contrast, the online detection phase is influenced only by c and h_s .

2) *Spatial Complexity*: The model parameters and gradient information primarily determine space complexity. During offline training, the space complexity of PatchST can be approximated as follows:

$$\begin{aligned} O_p + \partial O_p + O_p^a &= 2O_p + O_p^a = \underbrace{2O(p \cdot e)}_{\text{Embedding Layer}} \\ &+ \underbrace{2O(l \cdot (N \cdot e + 2e^2 + 2e))}_{\text{Transformer Layer}} + \underbrace{2O(e \cdot d_u)}_{\text{Dense Layer}} + \underbrace{2O(b)}_{\text{Biases}}. \end{aligned} \quad (19)$$

Accordingly, the space complexity of MKF-ADS can be approximated as follows:

$$\begin{aligned} O_m &= O_s + \partial O_s + O_p = 2O_s + O_p \\ &= \underbrace{2O(c^2 \cdot f_s)}_{\text{Conv1D}} + \underbrace{2O(s_l \cdot 2h_s)}_{\text{BiLSTM}} + \underbrace{2O(4h_s^2 + 2h_s)}_{\text{Attention Mechanism}} \\ &+ \underbrace{2O(2d_u * h_s)}_{\text{Dense Layer}} + \underbrace{2O(b)}_{\text{Biases}} + O_p. \end{aligned} \quad (20)$$

Similarly, during the training phase, the total space complexity of MKF-ADS is determined by N , e , c , and h_s , while the space complexity in the online detection phase is influenced by h_s and c . Due to fine pruning, these parameters of MKF-ADS are constrained to be sufficiently small, thereby exhibiting low temporal and spatial complexity. In other words, the MKF-IDS is lightweight and suitable for IVNs.

V. PERFORMANCE EVALUATION

In this section, we comprehensively evaluate the effectiveness and efficiency of MKF-ADS in preventing malicious attacks on vehicular networks.

A. Experiment Setup

1) *Hardware & Software*: To develop the MKF-ADS, we used PyTorch 1.13 in Python for model training. This training process was run on a server equipped with an Intel(R) Core(TM) i7-9500 U CPU @ 3.6 GHz with 64 GB of RAM and an RTX 3090 GPU. In the testing phase, the model's performance and effectiveness were evaluated on a real vehicle equipped with an NVIDIA Jetson AGX Xavier (16 GB).

2) *Metrics*: In the publicly available dataset, we consider each abnormal sequence as a positive case and each normal sequence as a negative one. Accordingly, we calculate four

statistical metrics: true positive (TP), true negative (TN), false positive (FP), and false negative (FN). Then, the precision (P), recall (R), and F1-Score are calculated as follows.

$$P = \frac{TP}{TP + FP}, R = \frac{TP}{TP + FN}, F1 = \frac{2 \times P \times R}{P + R}, \quad (21)$$

where high precision indicates the model's ability to accurately identify attacks, and stable recall represents that the model will not miss negatives. The F1-score represents the balance between precision and recall. Also, the error rate (ER) and false alarm rate (FAR), both critical measures of detection system performance, are reported and calculated as follows:

$$ER = \frac{FP + FN}{TP + TN + FP + FN}, FAR = \frac{FP}{FP + TN} \quad (22)$$

where ER represents the percentage of incorrect classifications relative to the total sample, and a lower FAR represents high-quality service from the IDS without constant false alarms.

3) *Implementation Details*: We employ grid-search to identify the optimal hyperparameters, including slide window size $T \in \{8, 16, 32, 64\}$, filter size $f \in \{8, 16, 32, 64\}$, hidden units $h \in \{4, 8, 16, 32\}$, dropout rate of 0.2, and batch size $b \in \{128, 256, 512, 1024\}$. For the teacher model (PatchST), all CAN IDs used a patch length of $P = 4$, a stride of $S = 1$, 8-dimensional embedding, and 2 transformer heads. The Adam optimizer was applied with a learning rate of $1e-2$, and training was conducted for 3000 epochs with early stopping.

4) *Baseline Models*: To deeply analyze the advantages of knowledge fusion, we compare MKF-ADS with prediction-based methods. These methods use only partial knowledge of MKF-ADS and are described as follows:

- *LSTM-P*: This model uses the LSTM network to extract temporal knowledge and predict a 64-bit physical value for the next time step [21].
- *LSTM-E*: This model improves on LSTM-P through optimization function, and its prediction target is a decimal physical value [22].
- *DeepConvGRU*: This model uses a depthwise separable convolutional layer and two extended RNN layers to extract spatial-temporal knowledge [42].
- *ConvLSTM-GNB*: This model integrates spatial-temporal knowledge with Gaussian Naive Bayes (GNB) under a supervised learning [43]. To ensure fairness, we only use its prediction module for anomaly detection.
- *CLAM*: This model leverages spatial-temporal attention knowledge to identify anomalies [24].

B. Detection Performance

To evaluate the performance of MKF-ADS, we build a multivariate time series for the same CAN ID. As described in Section IV-B, we first select key features using the “Read” algorithm. Then, we filter 7 out of 28 CAN IDs for evaluation based on the criteria that each must have more than 15 features and a periodicity of approximately 10 ms, as shown in Fig. 6(a). Fig. 6(b) shows the various signal channels of the selected CAN IDs, as determined by the boundary determination algorithm. For offline training, 80% of the dataset is used for training and

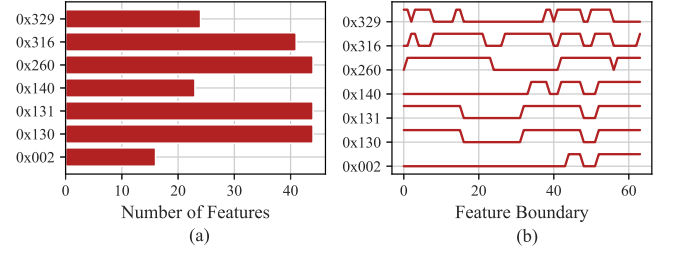


Fig. 6. The number of features and feature boundaries for candidate CAN IDs, where (a) shows feature selection based on the “Read” algorithm, and (b) displays the results of boundary determination.

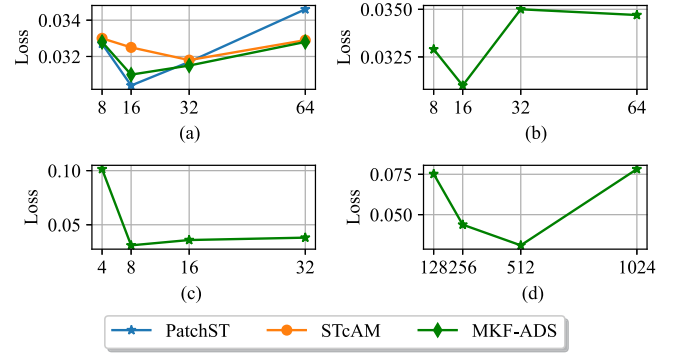


Fig. 7. Prediction deviation for different hyperparameters under grid search, where loss represents MAE.

the remaining 20% for testing. Unless otherwise specified, we use CAN ID 0x260 for evaluation.

Subsequently, we employ a grid search algorithm to identify the optimal hyperparameters, as shown in Fig. 7. In this step, we set the sliding window size of 16, which improves contextual and spatial-temporal knowledge modeling. In contrast, incorporating longer historical data introduces noise, leading to a significant decline in accuracy. The filter size and the number of hidden cells are set to 16 and 8, respectively, which not only yield the lowest MAE but also substantially reduce the model's parameter count. Notably, increasing the number of parameters only adds to inference complexity without any improvement in prediction accuracy. Additionally, a batch size of 512 is chosen to facilitate rapid convergence during the cross-fertilization of knowledge. After determining the optimal hyperparameters, we train MKF-ADS until convergence. The prediction results are shown in Fig. 8, where the predicted values closely align with the actual values. Compared to the method in [24], our model improves prediction performance in fine-grained variations of the CAN sequence. We attribute this improvement to the effective trend learning enabled by the knowledge fusion mechanism in MKF-ADS.

A well-trained ADS is expected to exhibit minimal prediction deviation under normal messages. Table II presents the prediction performance in comparison with baseline models. Evolving from temporal knowledge, spatial-temporal knowledge, and spatial-temporal attention knowledge to the proposed knowledge fusion framework, our model surpasses baselines.

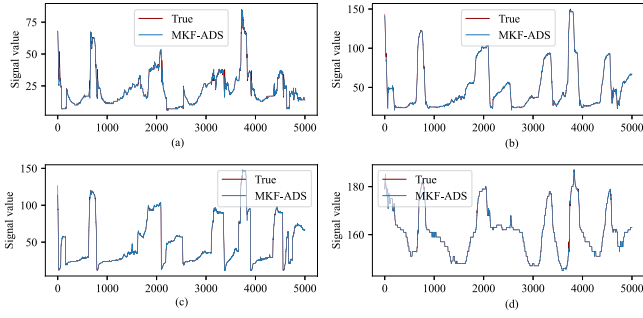


Fig. 8. The prediction results of the MKF-ADS model. The red line is the ground truth, and the blue line is the prediction result of the MKF-ADS.

TABLE II
PREDICTION DEVIATION OF THE MODEL COMPARED WITH BASELINE

Model	MAE	MAPE	RMSE
LSTM-P [22]	0.3824	-	0.7350
LSTM-E [23]	0.0565	0.6939	0.1387
DeepConvGRU [43]	0.0492	0.7355	0.1082
ConvLSTM-GNB [44]	0.0439	0.4648	0.1037
CLAM [25]	0.0391	0.2547	0.0905
PatchST	0.0314	0.1779	0.0858
STcAM	0.0325	0.1855	0.0955
MKF-ADS	0.0305	0.1992	0.0832

Notably, the LSTM-P performs the worst, followed by the LSTM-E due to the lack of knowledge fusion. The MAE and RMSE of DeepConvGRU and ConvLSTM-GNB show slight improvements when spatial features are incorporated. Among the baselines, CLAM achieves the best performance, with its attention mechanism enabling the model to focus on critical time steps, resulting in MAE and RMSE of only 0.0391 and 0.0905. Similarly, the MAPE follows the same trend, decreasing from 0.6939 to 0.2547. In contrast, the proposed STcAM component further reduces the prediction error while lowering model complexity, achieving an MAE of 0.0325 and a MAPE of 0.1855. Also, the PatchST model effectively models contextual knowledge, resulting in the lowest error. Thus, the proposed MKF-ADS achieves a promising prediction performance, with only 0.0305 on the MAE.

To verify the performance of MKF-ADS in anomaly detection, we simulate attack sequences in six attack scenarios, as discussed in Section III-B. These attacks disrupt communication between ECUs by mimicking their behavior and modifying the data fields of the CAN frame. Also, we vary the anomalous in the degrees of deviation from the normal value to analyze the proposed model capability for anomaly identification under the terms of coarse and fine-grained. Figs. 9 and 10 present the anomaly detection performance of the model in the coarse-grained and fine-grained, respectively. MKF-ADS can accurately detect anomalous messages when the attacker attempts a coarse-grained attack to significantly alter the state of the vehicle in a short period. In such cases, the CAN bus will receive alerts and discard abnormal messages. Specifically, Fig. 9(a) presents the detection results for the DoS attack. Due to the injection of a large number of malicious messages over a short period,

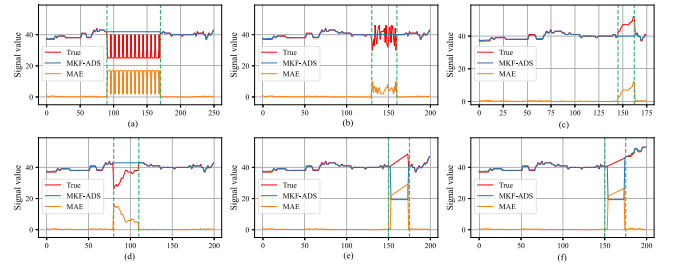


Fig. 9. Visualization of the detection of coarse-grained attacks, where the attacker injects anomalies that deviate significantly from normal values. The red line is the ground truth of the CAN sequence on the CAN bus, the blue line is the predicted value, and the yellow line is the anomaly score.

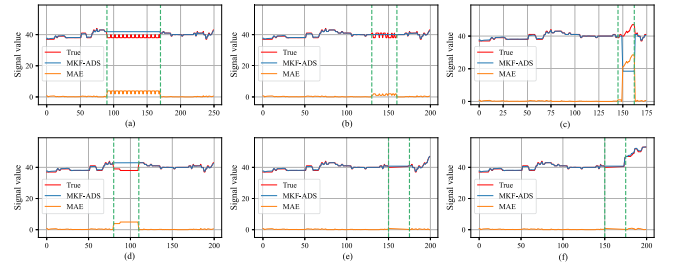


Fig. 10. Visualization of the detection of fine-grained attacks, where the attacker injects anomalies that closely resemble normal values. The red line is the ground truth of the CAN sequence on the CAN bus, the blue line is the predicted value, and the yellow line is the anomaly score.

MKF-ADS successfully identifies the anomaly as it frequently exceeds the threshold. In Fuzzy attacks, the attacker misleads the bus by injecting random signals. However, if the range of random signals becomes uncontrolled, MKF-ADS detects the anomaly. In suspension and replay attacks, compliant messages appear on the CAN bus at incorrect times. The model captures this timing disruption as anomalous deviations occur between predicted signals (based on the history window) and attack messages. In Fig. 9(e), the attacker purposefully modifies signal values to spoof the bus, for instance, by altering the gear speed. In contrast, the masquerade attack is more stealthy, removing normal messages before inserting malicious ones. Nevertheless, MKF-ADS easily detects such anomalies.

Next, we evaluate the proposed model at a fine-grained level, in which attacks are generated insidiously. In DoS attacks, numerous messages are injected within the boundaries of normal messages. Although fluctuation values drop dramatically, the pattern of injecting numerous messages still draws the model's attention. In Fuzzy attacks, the attacker generates random data within well-controlled bounds. During suspension attacks, the attacker deactivates the target ECU, disrupting a stable signal. Similarly, in a replay attack, the attacker records a segment of messages resembling normal signals and injects it later. Even though anomaly scores fluctuate slightly, MKF-ADS still recognizes anomalous intervals. Moreover, we simulate more insidious spoofing and masquerade attacks. In Fig. 10(e)–(f), the elaborately modified signals closely match the normal signals for several instances. However, the proposed model generates only

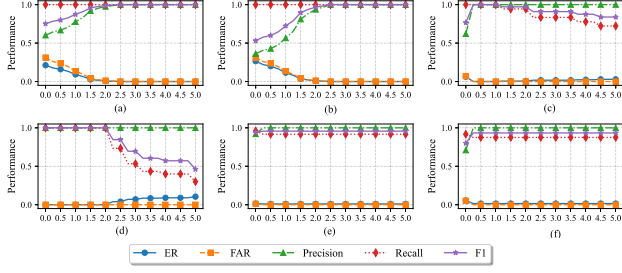


Fig. 11. Threshold determination for anomaly detection using a confidence interval-based approach.

TABLE III
COMPARISON WITH BASELINE IN SIX ATTACK SCENARIOS

Attack	Model	ER (%)	FAR (%)	R (%)	P (%)	F1 (%)
DoS ($K = 2.50, \tau = 0.77$)	LSTM-P [22]	7.25	9.50	90.9	95.0	92.9
	LSTM-E [23]	5.85	7.00	93.1	95.3	94.2
	DeepConvGRU [43]	3.35	4.00	96.0	97.3	96.6
	ConvLSTM-GNB [44]	2.00	2.90	100	94.1	96.9
	CLAM [25]	2.45	2.20	97.7	97.3	97.5
	MKF-ADS (Ours)	0.00	0.00	100	98.7	99.3
Fuzzy ($K = 2.50, \tau = 0.71$)	LSTM-P [22]	7.85	7.80	92.1	92.1	92.1
	LSTM-E [23]	5.87	6.80	93.2	95.0	94.1
	DeepConvGRU [43]	5.97	4.84	95.0	92.9	93.9
	ConvLSTM-GNB [44]	2.00	2.35	100	88.2	93.7
	CLAM [25]	4.32	4.84	96.2	95.2	95.6
	MKF-ADS (Ours)	0.50	0.50	100	96.7	98.3
Replay ($K = 0.25, \tau = 0.93$)	LSTM-P [22]	5.55	2.30	97.5	91.2	94.3
	LSTM-E [23]	5.42	5.05	94.9	94.2	94.5
	DeepConvGRU [43]	4.55	3.10	96.8	94.0	95.3
	ConvLSTM-GNB [44]	2.50	2.90	100	85.7	92.3
	CLAM [25]	4.67	5.05	94.9	95.8	95.3
	MKF-ADS (Ours)	1.76	1.97	98.6	96.1	97.3
Suspension ($K = 0.50, \tau = 0.82$)	LSTM-P [22]	7.10	7.30	93.1	92.7	92.9
	LSTM-E [23]	6.60	7.30	94.1	92.8	93.4
	DeepConvGRU [43]	7.70	8.50	93.1	91.6	92.3
	ConvLSTM-GNB [44]	0.50	0.00	94.4	100	97.1
	CLAM [25]	2.10	1.70	98.2	97.5	97.8
	MKF-ADS (Ours)	0.65	0.00	97.2	100	98.5
Spoofing ($K = 0.25, \tau = 1.04$)	LSTM-P [22]	8.75	9.20	90.8	91.7	91.3
	LSTM-E [23]	8.25	9.20	90.9	92.7	91.8
	DeepConvGRU [43]	7.75	8.20	91.8	92.7	92.3
	ConvLSTM-GNB [44]	2.00	0.00	83.3	100	90.9
	CLAM [25]	6.10	5.20	94.7	93.0	93.8
	MKF-ADS (Ours)	1.00	0.00	91.6	100	95.6
Masquerade ($K = 0.50, \tau = 1.04$)	LSTM-P [22]	10.10	9.40	90.4	89.2	89.8
	LSTM-E [23]	9.00	9.20	90.8	91.2	91.0
	DeepConvGRU [43]	8.20	8.30	91.7	91.9	91.8
	ConvLSTM-GNB [44]	7.00	3.00	89.0	96.7	92.7
	CLAM [25]	6.75	6.30	93.6	92.8	93.2
	MKF-ADS (Ours)	1.50	0.00	87.5	100	93.3

one false alarm on the attack interval. Thus, MKF-ADS maintains stable performance, even in fine-grained attack scenarios.

To evaluate the detection performance of MKF-ADS, we find the optimal threshold for six attack scenarios. In Fig. 11, we observe that the effect of K on detection performance is critical, i.e., smaller values of K increase the FAR, while larger values of K result in missed alarms. Thus, we build different values of K and thresholds in these attack scenarios. As shown in Table III, MKF-ADS outperforms baseline models. LSTM-P and LSTM-E, which rely solely on temporal knowledge, exhibit the worst performance, with the highest FAR across various attacks. With the introduction of spatial-temporal knowledge, the DeepConvGRU and ConvLSTM-GNB can detect more anomalies, while exhibiting poorer robustness. CLAM demonstrates the most stable performance among the baseline, due to

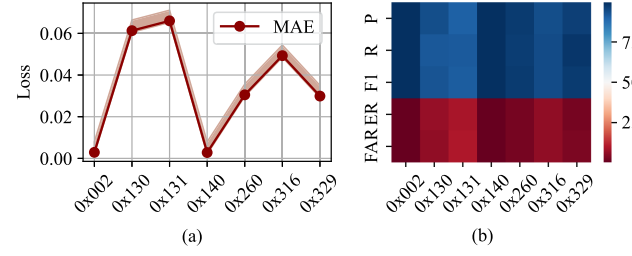


Fig. 12. Evaluation on all candidate CAN IDs, where (a) represents the test error, and (b) is the average performance on six attack scenarios.

attention mechanism. In contrast, the proposed model performs best on DoS attacks, achieving 98.7% precision and 100% recall. Also, MKF-ADS maintains 96.7% precision and 100% recall on fuzzy attacks, attributed to the random signals being completely distinct from the normal ones. In replay and suspension attacks, the proposed model can identify message injections that are inconsistent with timestamps but legitimate, achieving precision and recall of 96.1% vs. 100% and 98.6% vs. 97.2%, respectively. Significant performance degradation is observed in MKF-ADS for spoofing detection, as gradual load changes severely confuse the model, especially in masquerade attacks. Nevertheless, the proposed model remains highly competitive, with precision and recall of 100% vs. 100% and 91.6% vs. 87.5%. The model also achieves F1 scores of 99.3%, 98.3%, 97.3%, 98.5%, 95.6%, and 93.3%, respectively. Correspondingly, the error rate reflects the overall detection ability of MKF-ADS, only fluctuating between 0% and 1.76%. Moreover, maintaining a low FAR is crucial for vehicle status, and MKF-ADS reduces it to between 0% and 1.97%.

C. Generalization Performance

To evaluate the generalization performance of the proposed model, we report its prediction and detection abilities across all candidate CAN IDs. In Fig. 12, we find variations in the MAE across different CAN IDs, attributed to fluctuation in the data. For example, the periodic patterns for 0x002 and 0x004 result in a near-zero model error. The errors of CAN IDs similar to 0x260 maintained around 0.03. In contrast, 0x130 and 0x131 exhibit the worst prediction performance, with errors only reducing to around 0.06. Regarding detection performance, the precision, recall, and F1 scores converge to the blue color for all CAN IDs, indicating anomaly detection capability. Also, the errors converge to red, suggesting that the FAR and ER are within a controllable range.

Given a set of observations $X^{(i)} \in \mathbb{R}^{M \times T}$ for a CAN ID, we define the predict targets as $\hat{y}^{(i)} \in \mathbb{R}^{M \times L}$. Fig. 13 shows the evaluation of L from 1 to 5. First, the prediction error gradually increases as the number of prediction targets grows. For example, when predicting five-time steps, the MAE reached 0.054. Based on this observation, we set the corresponding thresholds for different prediction targets. Similarly, the proposed model maintains stable detection performance and achieves consistently low FAR and ER.

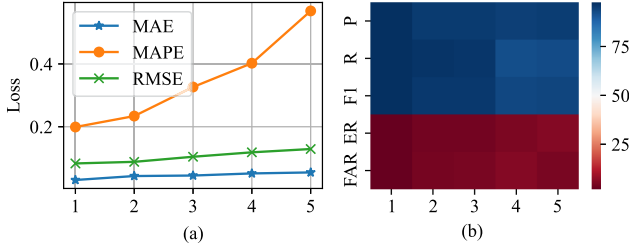


Fig. 13. Evaluation on time steps from 1 to 5, where (a) represents the test error, and (b) is the average performance on six attack scenarios.

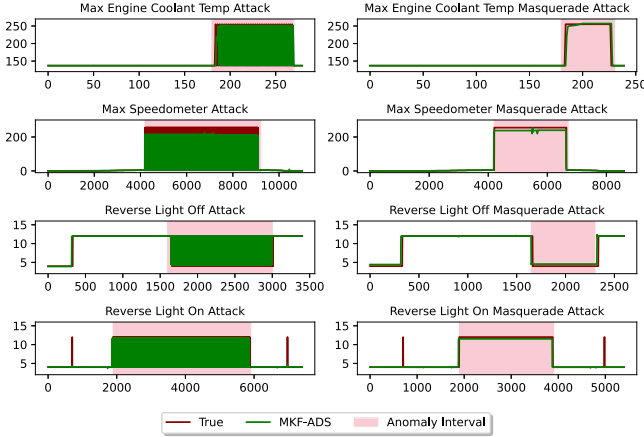


Fig. 14. Evaluation of MKF-ADS on the ROAD dataset with four spoofing attacks and four masquerade attacks, where the red line is the true value and the green line is the prediction of MKF-ADS. The x-axis represents the time steps and the y-axis represents the signal values.

Moreover, we present the generalization performance on the ROAD dataset. In this task, numerous attacks target spoofing and masquerade scenarios, which can seriously impact the physical state of vehicles. These effects include the accelerator failing, dashboard lights and headlights turning on, and the seat position shifting. For example, a spoof on 0x0D0 modifies the third bytes to '0 C' to change the status of the reverse light from off to on. Fig. 14 shows the prediction results for spoofing and masquerade attack scenarios, related to the Engine, Speedometer, and Light. Based on our observation, the vehicle state remains constant on the CAN bus for a short time. However, when the attacker injects messages that alter the vehicles' behavior (e.g., speed changed to 'FF'), MKF-ADS deviates significantly from the anomalous message during the attack interval due to only having the current signal. In short, the proposed model can identify attack periods based on specific thresholds and then alert the bus-layer.

D. Vehicle-Level Model Efficiency Evaluation

In this subsection, we compare and evaluate whether the model is suitable for IVNs based on the computation cost and time required for anomaly detection. Specifically, we set up a vehicle-level evaluation on a real vehicle (SAIC Roewe Marvel X), as shown in Fig. 15(a). First, we connect the PC to the vehicle's OBD-II port via the serial peripheral interface (SPI) and then record CAN logs using the CAN-Test software. The

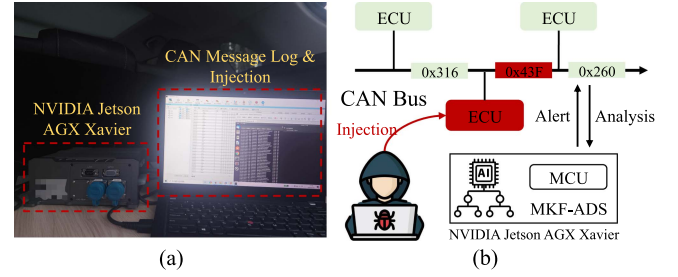


Fig. 15. Vehicle-level setup for model efficiency evaluation. (a) Equipment components, and (b) Connections between experimental components.

TABLE IV
COMPARISON OF COMPUTATION COST

Model	#Number of Parameters	FLOPs (M)	MACs (M)	Memory (MB)
LSTM-P [22]	8,857	7.0×10^{-2}	7.0×10^{-2}	0.04
LSTM-E [23]	3,481	2.2×10^{-2}	2.0×10^{-2}	0.02
DeepConvGRU [43]	627,429	20.83	20.80	2.64
CLAM [25]	22,308	6.9×10^{-1}	6.8×10^{-1}	0.12
ConvLSTM-GNB [44]	6,435	3.0×10^{-2}	3.0×10^{-2}	0.03
MKF-ADS (Ours)	1,952	1.9×10^{-2}	2.0×10^{-2}	0.01

TABLE V
COMPARISON OF TIME COST ON THE REAL VEHICLE

Vehicle	Model	DCT (ms)	DT (ms)	TT (ms)
SAIC Roewe Marvel x	LSTM-P [22]	7.75	11.42	19.17
	LSTM-E [23]	6.32	3.85	10.17
	DeepConvGRU [43]	6.51	5.98	12.49
	CLAM [25]	6.09	2.70	8.79
	ConvLSTM-GNB [44]	5.84	5.51	11.35
	MKF-ADS (Ours)	6.33	2.28	8.61

software also supports injecting attack messages into the vehicle, with attack rules consistent with the attack model discussed in Section III-B. Next, we deploy MKF-ADS on the vehicles' intelligent controller, powered by an NVIDIA Jetson Xavier (16 GB). The connections between the experiment equipment are shown in Fig. 15(b).

We report the computational cost in terms of the number of parameters, floating-point operations (FLOPs), multiply-accumulate operations (MACs), and memory usage, using the `thop` and `torchinfo` libraries. Table IV shows a comparison between the proposed model and existing studies. Clearly, MKF-ADS achieves knowledge fusion using only 1,952 parameters. In contrast, the parameter count of the baseline increases significantly depending on the type of incorporated knowledge. Furthermore, MKF-ADS requires FLOPs and MACs comparable to those of LSTM-based models. Notably, the proposed method consumes the least memory due to parameter pruning. Table V shows a time cost analysis comparing the proposed model with the baseline. In this table, DCT, DT, and TT represent the Data Collection Time, Detection Time, and Total Time for each intrusion detection. Based on our observations, the DCT depends on the observation window size during pre-processing, while DT is influenced by model complexity. In this case, we find that all models exhibit similar DCT costs, while the proposed method only requires 2.28 ms for anomaly detection. As a result,

the MKF-ADS achieves the lowest TT compared to the baseline, making it well-suited for IVNs.

VI. CONCLUSION AND FUTURE DIRECTIONS

In this paper, we propose MKF-ADS, a multi-knowledge fusion based anomaly detection model for the vehicular network. This model integrates a spatial-temporal correlation module with an attention mechanism (STcAM) and patch-based sparse transformer (PatchST) module. Furthermore, the model introduces cross-knowledge distillation into STcAM, treating it as a student model to learn intrinsic knowledge and cross-mimic the capabilities of PatchST. Thus, the model achieves a knowledge-fused at the feature level and is well-suited for resource-limited IVN environments. Comparing our method with the baseline in this field, the prediction error and detection capability are improved and robust.

However, several issues in ADS still need to be addressed before achieving practical application in IVNs. First, we will explore how to mitigate noise effects during knowledge fusion. A promising direction is to use interpretable analysis for precise knowledge fusion. Also, we aim to enhance forecasting performance across multiple time steps, enabling early-phase attack detection. To reduce model complexity, we plan to develop an integrated ADS for all CAN IDs, facilitating secure interactions between ECUs.

REFERENCES

- [1] H. M. Song and H. K. Kim, "Self-supervised anomaly detection for in-vehicle network using noised pseudo normal data," *IEEE Trans. Veh. Technol.*, vol. 70, no. 2, pp. 1098–1108, Feb. 2021.
- [2] K. Agrawal, T. Alladi, A. Agrawal, V. Chamola, and A. Benslimane, "NovelADS: A novel anomaly detection system for intra-vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 22596–22606, Nov. 2022.
- [3] M. Han, P. Cheng, and S. Ma, "PPM-InVIDS: Privacy protection model for in-vehicle intrusion detection system based complex-valued neural network," *Veh. Commun.*, vol. 31, 2021, Art. no. 100374.
- [4] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based intrusion detection system for in-vehicle network," in *Proc. 16th Annu. Conf. Privacy, Secur. Trust*, 2018, pp. 1–6.
- [5] H. Liu et al., "Blockchain and federated learning for collaborative intrusion detection in vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 6073–6084, Jun. 2021.
- [6] K. Pawelec, R. A. Bridges, and F. L. Combs, "Towards a can IDS based on a neural network data field predictor," in *Proc. ACM Workshop Automot. Cybersecurity*, 2019, pp. 31–34.
- [7] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A multitiered hybrid intrusion detection system for internet of vehicles," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 616–632, Jan. 2022.
- [8] A. Boualouache and T. Engel, "A survey on machine learning-based misbehavior detection systems for 5G and beyond vehicular networks," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 2, pp. 1128–1172, Secondquarter 2023.
- [9] J. Ashraf, A. D. Bakhshi, N. Moustafa, H. Khurshid, A. Javed, and A. Beheshti, "Novel deep learning-enabled LSTM autoencoder architecture for discovering anomalous events from intelligent transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4507–4518, Jul. 2021.
- [10] L. Wang et al., "Periodic event-triggered fault detection for safe platooning control of intelligent and connected vehicles," *IEEE Trans. Veh. Technol.*, vol. 73, no. 4, pp. 5064–5077, Apr. 2024.
- [11] K. H. Park et al., "Unsupervised malicious domain detection with less labeling effort," *Comput. Secur.*, vol. 116, 2022, Art. no. 102662.
- [12] J. R. V. Solaas, N. Tuptuk, and E. Mariconti, "Systematic review: Anomaly detection in connected and autonomous vehicles," 2024, *arXiv:2405.02731*.
- [13] H. Luo et al., "ESIA: An efficient and stable identity authentication for internet of vehicles," *IEEE Trans. Veh. Technol.*, vol. 73, no. 4, pp. 5602–5615, Apr. 2024.
- [14] O. Schell and M. Kneib, "Valid: Voltage-based lightweight intrusion detection for the controller area network," in *Proc. IEEE 19th Int. Conf. Trust, Secur. Privacy Comput. Commun.*, 2020, pp. 225–232.
- [15] M. Foruhandeh, Y. Man, R. Gerdes, M. Li, and T. Chantem, "Simple: Single-frame based physical layer identification for intrusion detection and prevention on in-vehicle networks," in *Proc. 35th Annu. Comput. Secur. Appl. Conf.*, 2019, pp. 229–244.
- [16] M. Kneib, O. Schell, and C. Huth, "EASI: Edge-based sender identification on resource-constrained platforms for automotive networks," in *Proc. NDSS*, 2020, pp. 1–16.
- [17] P. Cheng, M. Han, and G. Liu, "DESC-IDS: Towards an efficient real-time automotive intrusion detection system based on deep evolving stream clustering," *Future Gener. Comput. Syst.*, vol. 140, pp. 266–281, 2023.
- [18] F. Erlacher and F. Dressler, "On high-speed flow-based intrusion detection using snort-compatible signatures," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 495–506, Jan./Feb. 2022.
- [19] Z. Wu, H. Zhang, P. Wang, and Z. Sun, "RTIDS: A robust transformer-based approach for intrusion detection system," *IEEE Access*, vol. 10, pp. 64375–64387, 2022.
- [20] C. Liu, T. Nie, Y. Du, J. Cao, D. Wu, and F. Li, "A response-type road anomaly detection and evaluation method for steady driving of automated vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 21984–21995, Nov. 2022.
- [21] A. Taylor, S. Leblanc, and N. Japkowicz, "Anomaly detection in automobile control network data with long short-term memory networks," in *Proc. IEEE Int. Conf. Data Sci. Adv. Analytics*, 2016, pp. 130–139.
- [22] H. Qin, M. Yan, and H. Ji, "Application of controller area network (CAN) bus anomaly detection based on time series prediction," *Veh. Commun.*, vol. 27, 2021, Art. no. 100291.
- [23] M. Hanselmann, T. Strauss, K. Dormann, and H. Ulmer, "CANet: An unsupervised intrusion detection system for high dimensional CAN bus data," *IEEE Access*, vol. 8, pp. 58194–58205, 2020.
- [24] H. Sun, M. Chen, J. Weng, Z. Liu, and G. Geng, "Anomaly detection for in-vehicle network using CNN-LSTM with attention mechanism," *IEEE Trans. Veh. Technol.*, vol. 70, no. 10, pp. 10880–10893, Oct. 2021.
- [25] N. Alkhatib, M. Mushtaq, H. Ghauch, and J. -L. Danger, "CAN-BERT do it? Controller area network intrusion detection system based on BERT language model," in *Proc. IEEE/ACS 19th Int. Conf. Comput. Syst. Appl.*, 2022, pp. 1–8.
- [26] M. Nam, S. Park, and D. S. Kim, "Intrusion detection method using bi-directional GPT for in-vehicle controller area networks," *IEEE Access*, vol. 9, pp. 124931–124944, 2021.
- [27] M. Marchetti and D. Stabili, "Read: Reverse engineering of automotive data frames," *IEEE Trans. Inf. Forensics Secur.*, vol. 14, no. 4, pp. 1083–1097, Apr. 2019.
- [28] M. Muter and N. Asaj, "Entropy-based anomaly detection for in-vehicle networks," in *Proc. IEEE Intell. Veh. Symp.*, 2011, pp. 1110–1115.
- [29] H. M. Song, H. R. Kim, and H. K. Kim, "Intrusion detection system based on the analysis of time intervals of can messages for in-vehicle network," in *Proc. Int. Conf. Inf. Netw.*, 2016, pp. 63–68.
- [30] H. Olufowobi, C. Young, J. Zambreno, and G. Bloom, "SAIDuCANT: Specification-based automotive intrusion detection using controller area network (CAN) timing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 1484–1494, Feb. 2020.
- [31] Z. Zhang, Y. Cao, Z. Cui, W. Zhang, and J. Chen, "A many-objective optimization based intelligent intrusion detection algorithm for enhancing security of vehicular networks in 6G," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 5234–5243, Jun. 2021.
- [32] V. Tanksale, "Intrusion detection for controller area network using support vector machines," in *Proc. IEEE 16th Int. Conf. Mobile Ad Hoc Sensor Syst. Workshops*, 2019, pp. 121–126.
- [33] M. R. Dey, M. Patra, and P. Mishra, "Efficient detection and localization of DoS attacks in heterogeneous vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 5, pp. 5597–5611, May 2023.
- [34] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," *Veh. Commun.*, vol. 21, 2020, Art. no. 100198.
- [35] Y. Xun, J. Qin, and J. Liu, "Deep learning enhanced driving behavior evaluation based on vehicle-edge-cloud architecture," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 6172–6177, Jun. 2021.

- [36] W. Lo, H. Alqahtani, K. Thakur, A. Almadhor, S. Chander, and G. Kumar, "A hybrid deep learning based intrusion detection system using spatial-temporal representation of in-vehicle network traffic," *Veh. Commun.*, vol. 35, 2022, Art. no. 100471.
- [37] P. Cheng, M. Han, A. Li, and F. Zhang, "STC-IDS: Spatial-temporal correlation feature analyzing based intrusion detection system for intelligent connected vehicles," *Int. J. Intell. Syst.*, vol. 37, no. 11, pp. 9532–9561, 2022.
- [38] A. R. Javed, S. U. Rehman, M. U. Khan, M. Alazab, and T. Reddy, "CANintelliIDS: Detecting in-vehicle intrusion attacks on a controller area network using CNN and attention-based GRU," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1456–1466, Apr.–Jun. 2021.
- [39] S. Jeong, S. Lee, H. Lee, and H. K. Kim, "X-CANIDS: Signal-aware explainable intrusion detection system for controller area network-based in-vehicle network," *IEEE Trans. Veh. Technol.*, vol. 73, no. 3, pp. 3230–3246, Mar. 2024.
- [40] S. Tariq, S. Lee, and S. S. Woo, "CANTransfer: Transfer learning based intrusion detection on a controller area network using convolutional LSTM network," in *Proc. 35th Annu. ACM Symp. Appl. Comput.*, 2020, pp. 1048–1055.
- [41] H. Zhang, K. Zeng, and S. Lin, "Federated graph neural network for fast anomaly detection in controller area networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 1566–1579, 2023.
- [42] J. Zhang et al., "A deep learning framework for driving behavior identification on in-vehicle CAN-BUS sensor data," *Sensors*, vol. 19, no. 6, 2019, Art. no. 1356.
- [43] P. Mansourian, N. Zhang, A. Jaekel, and M. Kneppers, "Deep learning-based anomaly detection for connected autonomous vehicles using spatiotemporal information," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 16006–16017, Dec. 2023.
- [44] M. A. Shawky, M. Bottarelli, G. Epiphaniou, and P. Karadimas, "An efficient cross-layer authentication scheme for secure communication in vehicular ad-hoc networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 7, pp. 8738–8754, Jul. 2023.
- [45] L. Dariz, M. Selvatici, M. Ruggeri, G. Costantino, and F. Martinelli, "Trade-off analysis of safety and security in CAN bus communication," in *Proc. 5th IEEE Int. Conf. Models Technol. Intell. Transp. Syst.*, 2017, pp. 226–231.
- [46] W. A. Farag, "CANTrack: Enhancing automotive CAN bus security using intuitive encryption algorithms," in *Proc. 7th Int. Conf. Model., Simul., Appl. Optim.*, 2017, pp. 1–5.
- [47] G. Bella, P. Biondi, G. Costantino, and I. Matteucci, "TOUCAN: A protocol to secure controller area network," in *Proc. ACM Workshop Automot. Cybersecurity*, 2019, pp. 3–8.
- [48] T. Alladi, S. Chakravarty, V. Chamola, and M. Guizani, "A lightweight authentication and attestation scheme for in-transit vehicles in IoV scenario," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 14188–14197, Dec. 2020.
- [49] J. Cui et al., "Lightweight encryption and authentication for controller area network of autonomous vehicles," *IEEE Trans. Veh. Technol.*, vol. 72, no. 11, pp. 14756–14770, Nov. 2023.
- [50] Y. Xun, Z. Deng, J. Liu, and Y. Zhao, "Side channel analysis: A novel intrusion detection system based on vehicle voltage signals," *IEEE Trans. Veh. Technol.*, vol. 72, no. 6, pp. 7240–7250, Jun. 2023.
- [51] I. Studnia, E. Alata, V. Nicomette, M. Kaâniche, and Y. Laarouchi, "A language-based intrusion detection approach for automotive embedded networks," *Int. J. Embedded Syst.*, vol. 10, no. 1, pp. 1–12, 2018.
- [52] A. H. Sodhro, G. H. Sodhro, M. Guizani, S. Pirbhulal, and A. Boukerche, "AI-enabled reliable channel modeling architecture for fog computing vehicular networks," *IEEE Wireless Commun.*, vol. 27, no. 2, pp. 14–21, Apr. 2020.
- [53] L. Sun, J. Liang, and G. Muhammad, "Distillate a sparse-meta time series classifier for open radio access network-based cellular vehicle-to-everything," *IEEE Trans. Veh. Technol.*, vol. 73, no. 7, pp. 9262–9271, Jul. 2024.
- [54] M. E. Verma et al., "Addressing the lack of comparability & testing in CAN intrusion detection research: A comprehensive guide to CAN IDS data & introduction of the ROAD dataset," 2020, *arXiv:2012.14600*.
- [55] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *Proc. 11th Int. Conf. Learn. Representations*, 2022.
- [56] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log anomaly detection via BERT," in *Proc. Int. Joint Conf. Neural Netw.*, 2021, pp. 1–8.
- [57] J. Wang, Y. Chen, Z. Zheng, X. Li, M. -M. Cheng, and Q. Hou, "CrossKD: Cross-head knowledge distillation for object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 16520–16530.



Pengzhou Cheng received the M.S. degree from the Department of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, China, in 2022. He is currently working toward the Ph.D. degree with the Department of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China. He is also co-supervised by Prof. Fengwei Zhang with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China. His research interests include artificial intelligence security, backdoor attacks and defenses, cybersecurity, machine learning, deep learning, and intrusion detection systems.



Shouxuan Liu is currently working toward the B.S. degree in electrical engineering and physics with the University of Michigan, Ann Arbor, MI, USA. His research interests include semiconductor devices, machine learning, and large-scale integrated photonics.



Zongru Wu received the B.S. degree from the School of Cyber Science and Engineering, Wuhan University, Hubei, China, in 2022. He is currently working toward the Ph.D. degree with the School of Cyber Science and Engineering, Shanghai Jiao Tong University, Shanghai, China. His research interests include artificial intelligence security, backdoor attack and countermeasures, cybersecurity, machine learning, and deep learning.



Lixing Tan received the Ph.D. degree in computer science from Nanjing University of Aeronautics and Astronautics, Nanjing, China. He is currently a Lecturer with the College of Information Engineering, Taizhou University, Taizhou, China. He has authored or coauthored several papers in internationally renowned journals, such as *Artificial Intelligence* and *International Journal of Approximate Reasoning*. His research interests include explainable artificial intelligence, knowledge representation, and reasoning.



Gongshen Liu received the Ph.D. degree from the Department of Computer Science, Shanghai Jiao Tong University, Shanghai, China. He is currently a Professor with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University. His research interests include natural language processing, machine learning, and artificial intelligence security.